ASSIGNMENT 3.1

2303A51930

BATCH-30

Question 1: Zero-Shot Prompting (Palindrome Number Program)
Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.
Task:
• Record the AI-generated code.
• Test the code with multiple inputs.
• Identify any logical errors or missing edge-case handling.

CODE:

```
def is_palindrome(number):
    # Convert input to string and remove any potential whitespace
    str_num = str(number).strip()
    # Check if string is equal to its reverse
    return str_num == str_num[::-1]

# Get user input
user_input = input("Enter a number to check: ")

if is_palindrome(user_input):
    print(f'{user_input} is a palindrome.')
else:
    print(f'{user_input} is not a palindrome.')
```

OUTPUT:

```
Enter a number to check: 115
115 is not a palindrome.
```

```
•••  Enter a number to check: 626
     626 is a palindrome.
```

Question 2: One-Shot Prompting (Factorial Calculation)
Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.
Example:
Input: 5 → Output: 120
Task:
• Compare the generated code with a zero-shot solution.

• Examine improvements in clarity and correctness.

CODE:

```
def factorial(n):
    if n < 0:
        return "Factorial is not defined for negative numbers"
    elif n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

# Get user input
try:
    user_num = int(input("Enter a number to find its factorial: "))
    result = factorial(user_num)
    print(f"The factorial of {user_num} is {result}")
except ValueError:
    print("Please enter a valid integer.")
```

OUPUT:

```
•••  Enter a number to find its factorial: 10
     The factorial of 10 is 3628800
```

Question 3: Few-Shot Prompting (Armstrong Number Check)
Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.
Examples:
• Input: 153 → Output: Armstrong Number
• Input: 370 → Output: Armstrong Number
• Input: 123 → Output: Not an Armstrong Number
Task:
• Analyze how multiple examples influence code structure and accuracy.
• Test the function with boundary values and invalid inputs.

CODE:

```
def is_armstrong(number):
    # Convert to string to easily iterate through digits
    num_str = str(number).strip()
    num_digits = len(num_str)

    # Calculate the sum of digits raised to the power of num_digits
    try:
        total_sum = sum(int(digit) ** num_digits for digit in num_str)
```

```
        return total_sum == int(num_str)
    except ValueError:
        return False

# Get user input
user_input = input("Enter a number to check if it is an Armstrong number: ")

try:
    val = int(user_input)
    if is_armstrong(val):
        print(f"{val} is an Armstrong number.")
    else:
        print(f"{val} is not an Armstrong number.")
except ValueError:
    print("Please enter a valid integer.")
```

OUTPUT:

```
Enter a number to check if it is an Armstrong number: 153
153 is an Armstrong number.
```

```
•••  Enter a number to check if it is an Armstrong number: 370
     370 is an Armstrong number.
```

```
Enter a number to check if it is an Armstrong number: 123
123 is not an Armstrong number.
```

Question 4: Context-Managed Prompting (Optimized Number Classification)
Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.
Task:
• Ensure proper input validation.
• Optimize the logic for efficiency.
• Compare the output with earlier prompting strategies.

CODE:

```
import math

def check_number_type(n):
    if n <= 1:
        return "neither prime nor composite"

    # Check for divisors from 2 up to the square root of n
    is_prime = True
```

```
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            is_prime = False
            break

    if is_prime:
        return "a prime number"
    else:
        return "a composite number"

# Get user input
try:
    user_input = input("Enter a positive integer: ")
    val = int(user_input)

    if val < 0:
        print("Please enter a non-negative integer.")
    else:
        result = check_number_type(val)
        print(f"{val} is {result}.")
except ValueError:
    print("Please enter a valid integer.")
```

OUTPUT:

```
Enter a positive integer: 50
50 is a composite number.
```

```
Enter a positive integer: 11
11 is a prime number.
```

```
Enter a positive integer: 0
0 is neither prime nor composite.
```

Question 5: Zero-Shot Prompting (Perfect Number Check)
Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.
Task:
• Record the AI-generated code.
• Test the program with multiple inputs.
• Identify any missing conditions or inefficiencies in the logic.

CODE:

```
def is_perfect_number(n):
    if n <= 0:
        return False

    # Find divisors and sum them
    divisors_sum = 0
    for i in range(1, n):
        if n % i == 0:
            divisors_sum += i
```

```
    return divisors_sum == n

# Get user input
try:
    user_input = input("Enter a number to check if it is a perfect number: ")
    val = int(user_input)

    if val <= 0:
        print("Please enter a positive integer.")
    else:
        if is_perfect_number(val):
            print(f"{val} is a perfect number.")
        else:
            print(f"{val} is not a perfect number.")
except ValueError:
    print("Please enter a valid integer.")
```

OUTPUT:

```
Enter a number to check if it is a perfect number: 24
24 is not a perfect number.
```

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

- Input: 8 → Output: Even
- Input: 15 → Output: Odd
- Input: 0 → Output: Even

Task:

- Analyze how examples improve input handling and output clarity.
- Test the program with negative numbers and non-integer inputs.

CODE:

```
def is_perfect_number(n):
    if n <= 0:
        return False

    # Find divisors and sum them
    divisors_sum = 0
    for i in range(1, n):
        if n % i == 0:
            divisors_sum += i

    return divisors_sum == n

# Get user input
```

```
try:
    user_input = input("Enter a number to check if it is a perfect number: ")
    val = int(user_input)

    if val <= 0:
        print("Please enter a positive integer.")
    else:
        if is_perfect_number(val):
            print(f"{val} is a perfect number.")
        else:
            print(f"{val} is not a perfect number.")
except ValueError:
    print("Please enter a valid integer.")
```

OUTPUT:

```
Enter a number to check if it is a perfect number: 24
24 is not a perfect number.
```